

Preorder Campaigns — Design Spec

Date: 2026-08-07 **Status:** Approved design — ready for implementation planning **Owner:** AfroTransact **Scope:** Full-stack (storefront `afrotransact-v2-ui` , backend `refined/services`)

1. Problem & Goal

Customers who buy **perishables in bulk** (tomatoes, bell peppers, yams, etc.) aren't served today: AfroTransact can't stock perishable goods in bulk without spoilage risk. A competitor offers preorder; we want to do it better.

Goal: Let admin open a time-boxed **preorder campaign** of perishable products. Customers preorder specific sizes and **pay upfront**. Admin sees **aggregated demand** so they know exactly what to buy at market, then distributes to customers on a set date — so AfroTransact never stores perishables.

Success looks like: admin opens a campaign → customers preorder & pay → admin gets a clean "shopping list" + "distribution list" → admin buys the exact demand and delivers → no inventory held, no spoilage.

2. Requirements (decided)

Decision	Choice
Preorder unit	Campaign — admin opens/closes it; entry point + dedicated page appear only while open
Timeline	Campaign-level — one <code>orderByAt</code> + one <code>distributionDate</code> ; all items share it
Concurrency	One campaign open at a time (v1 constraint)
Payment	Charge full amount now (reuses existing charge-immediately Stripe flow)
Delivery fee	One flat fee per preorder order (all items → one address → one fee), from campaign config
Tax	Campaign config override (<code>exempt</code> / <code>flat_rate</code> / <code>inherit</code>)
Cart	Separate preorder checkout — preorder items never mix with the regular cart
Product unit	Fixed sizes , each with a campaign-specific price (reuses product variants)
Refunds	Full cancel + partial (cancel a whole preorder, or refund N of M sourced units)
Operator	House/admin-operated — AfroTransact sources & distributes (no seller split)
Checkout config	Normal checkout flow, but campaign config overrides default tax/delivery/zone config

3. Architecture Approach

Ride the money rails, own the preorder concern. Preorder items flow through the existing cart → checkout-session → Stripe PaymentIntent → order → refund rails **unchanged**, so charging and refunding are reused, not rebuilt. Preorder gets a **first-class campaign entity** (typed timeline, sizes, config overrides, lifecycle) rather than flags scattered in jsonb.

Service ownership:

- **product-catalog** owns the campaign catalog (`PreorderCampaign` , `PreorderCampaignItem`) — it's fundamentally a curated set of products for a window.
- **order** owns preorder orders, demand aggregation, and fulfillment — reuses `CheckoutSession` → `materializePaidOrder` (event-driven on `payment.completed`).
- **payment** is unchanged (charge-now; full/partial refund already supported).
- **storefront** renders the `/preorder` page, the separate preorder cart, and the admin campaign UI.

4. Data Model

4.1 `PreorderCampaign` (product-catalog service)

Field	Type	Notes
<code>id</code>	uuid	PK
<code>name</code>	string	e.g. "This Week's Fresh Produce"
<code>slug</code>	string	for the <code>/preorder</code> URL / references
<code>status</code>	enum	<code>draft</code> → <code>open</code> → <code>closed</code> → <code>fulfilled</code> → <code>archived</code>
<code>orderByAt</code>	timestampz	preorder deadline; auto-close trigger
<code>distributionDate</code>	date	when customers receive items
<code>flatDeliveryFeeCents</code>	int	charged once per preorder order
<code>taxMode</code>	enum	<code>exempt</code> <code>flat_rate</code> <code>inherit</code>
<code>taxRateBps</code>	int, nullable	required when <code>taxMode = flat_rate</code>
<code>notes</code>	text, nullable	internal
<code>createdAt</code> / <code>updatedAt</code>	timestampz	

Invariant: at most one campaign in `open` status at any time (enforced in service + a partial unique index).

4.2 `PreorderCampaignItem` (product-catalog service)

Field	Type	Notes
<code>id</code>	uuid	PK
<code>campaignId</code>	uuid	FK → campaign
<code>productId</code>	uuid	a house product
<code>variantId</code>	uuid	the offered size (reuses product variants)
<code>priceCents</code>	int	campaign-specific price for this size (perishable prices change per campaign)
<code>available</code>	bool	admin can pull a size mid-campaign
<code>displayOrder</code>	int	ordering on the page

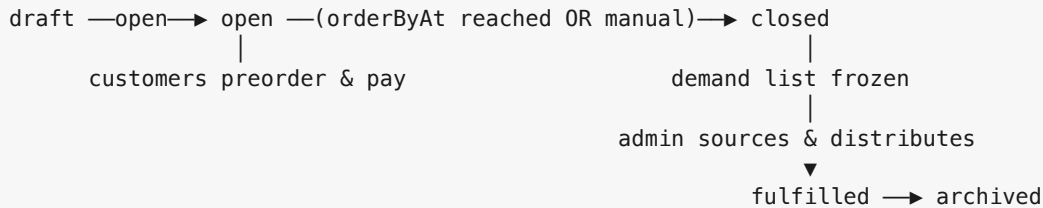
A product may appear as several rows (one per offered size). Price lives here, **not** on the product default.

4.3 Order tagging (order service)

Preorder orders reuse `Order` / `SubOrder` / `OrderItem` with additions:

- `Order.preorderCampaignId` (uuid, nullable) — set for preorder orders
- `Order.distributionDate` (date, nullable) — copied from the campaign at checkout
- `Order.isPreorder` (bool) — convenience marker/badge
- Single **house** `SubOrder` (no per-seller split; admin fulfills)

5. Campaign Lifecycle



- **draft** — admin builds it (products, sizes, prices, config). Not visible to customers.
- **open** — entry point + `/preorder` page live; customers preorder and pay. Opening enforces the one-open-at-a-time rule.
- **closed** — auto at `orderByAt` (or manual); no new preorders; demand list is final.
- **fulfilled** — admin has sourced and distributed; per-order fulfillment marked.
- **archived** — closed out for history.

6. Storefront (Customer) Flow

1. **Entry point** — while a campaign is `open`, a "Preorder" nav item + a homepage banner appear. The banner is a **dedicated preorder component gated on "a campaign is open"** (not the promotions/ticker scheduler — avoids coupling preorder visibility to the promo system). Absent when no campaign is open.
2. **`/preorder` page** — campaign header (name + "Order by Tue 6pm · Delivered Thu"), then item cards: product + size options with **campaign prices** + quantity picker. Closed/empty state: "No preorder is open right now."
3. **Preorder cart** — a **separate cart** from the regular one (cannot mix). Shows subtotal, the **flat delivery fee**, tax per campaign `taxMode`, and a persistent "Arrives [`distributionDate`]" banner.
4. **Checkout** — the **normal** address → pay flow. Totals resolve from **campaign config** (flat fee replaces the shipping quote; tax override applies). **Charged in full now.**
5. **Confirmation & tracking** — "Preorder confirmed — arriving [`distributionDate`]." Appears in order history with a **Preorder badge** + distribution date. Full/partial refunds notify the customer.

7. Admin Flow

1. **`admin/preorders`** — campaigns list (status, order-by / distribution dates, # preorders, total value).
2. **Create / edit campaign** — name, `orderByAt`, `distributionDate`, config overrides (flat delivery fee, tax mode/rate); add products → pick offered **sizes** → set **campaign price** per size; toggle availability.
3. **Open / close** — "Open" makes the button + page live; auto-closes at `orderByAt` or manually. One-open-at-a-time enforced.
4. **Demand view (core value)** — per campaign, two rollups over **paid** preorder orders:
 - **Shopping list**: product × size → total units + line value ("Tomatoes — 20kg crate ×50; 10kg basket ×30"), grand total. **CSV export** for the market run.

- **Distribution list:** per customer → items + delivery address, for the drop-off run. **CSV export.**
5. **Fulfillment** — mark distributed / delivered (reuses sub-order fulfillment statuses).
 6. **Cancel & refund** — cancel a whole campaign (refund everyone), cancel one customer's preorder (full refund), or **partial refund** (reduce sourced units → refund the difference).
-

8. Backend Mechanics

8.1 Checkout config override

- The preorder checkout is tagged with `preorderCampaignId` on the `CheckoutSession`.
- During quote/checkout, the order service detects a preorder session and **bypasses** the normal zone/tax resolution (`shippingQuotesForCart` platform-fallback branch), substituting the campaign's `flatDeliveryFeeCents` (charged once) and `taxMode / taxRateBps`.
- Everything else (address capture, `PaymentIntent` creation, `materializePaidOrder` on `payment.completed`) is unchanged.

8.2 Demand aggregation endpoint (order service)

`GET /api/v1/admin/preorders/campaigns/{campaignId}/demand` → returns:

- `shoppingList[]`: { `productId`, `productTitle`, `variantId`, `sizeLabel`, `units`, `unitPriceCents`, `lineTotalCents` }
- `distributionList[]`: { `customer`, `address`, `items[]`: { `productTitle`, `sizeLabel`, `quantity` } }
- `totals`: { `orders`, `unitsTotal`, `grossCents` }

Reads paid orders where `preorderCampaignId = {id}`. Backs both the admin demand view and CSV exports.

8.3 Order lifecycle (preorder path)

- On `payment.completed`, `materializePaidOrder` creates the order with `isPreorder = true`, `preorderCampaignId`, `distributionDate`, and a house sub-order at fulfillment status `preorder_confirmed`.
- Distribution reuses existing sub-order statuses: `preorder_confirmed` → `out_for_delivery` → `delivered` (and `cancelled / refunded`).

8.4 Payment & refunds

- **Charge:** existing Stripe `PaymentIntent` (immediate capture) — no provider change.
 - **Full refund:** existing refund path on cancel (whole preorder or whole campaign).
 - **Partial refund:** admin reduces sourced units on an order item; system refunds $(\text{orderedUnits} - \text{sourcedUnits}) \times \text{unitPrice}$, updates the item, and notifies the customer. **The flat delivery fee is retained on partial refunds** (one delivery still happens); it is refunded **only when the entire order is cancelled**.
-

9. Edge Cases

- **Campaign closes with a paid preorder mid-checkout** — checkout sessions already created before `orderByAt` may complete within a short grace window; new sessions are rejected once `closed`.
 - **Admin tries to open a second campaign** — blocked with a clear message; must close the current one first.
 - **Size pulled mid-campaign** (`available=false`) — hidden from new preorders; existing paid preorders for it stand until admin cancels/refunds.
 - **Customer cancels?** — v1: **no self-service cancel**; refunds are admin-initiated (matches house-operated model). Revisit later.
 - **Distribution date passes without fulfillment** — order stays `preorder_confirmed`; surfaced in admin as overdue.
-

10. Out of Scope (v1 — YAGNI)

- Multiple simultaneously-open campaigns.
 - Per-product timelines (campaign-level only).
 - Minimum-demand auto-cancel thresholds (admin decides manually).
 - Seller-created preorder products (house-operated only).
 - Customer self-service cancellation.
 - Deposit / authorize-and-capture payment models (charge-now only).
 - Auto-allocation on shortfall (admin does partial refunds manually).
 - Recurring/auto-scheduled campaigns.
-

11. Assumptions & Resolved Decisions

- **Assumption:** preorder products are **house products** created for the campaign; campaign price is authoritative over any product default price.
- **Resolved — partial-refund delivery fee:** the flat delivery fee is **kept on partial refunds** (one delivery still occurs) and **refunded only on full order cancellation**. (See §8.4.)
- **Resolved — preorder banner:** a **dedicated preorder banner component gated on "a campaign is open"**, independent of the promotions/ticker scheduler. (See §6.)